

Kod Sağlamaştırma: C/C++

CEN429 Güvenli Programlama — Hafta 4

Dr. Öğr. Üyesi Uğur CORUH · 09.10.2026

Bugünün planı (3 saat)

Saat	Bölüm	Konu
1	0–2	Temel kavramlar · sağlamlaştırma katmanları · SEI CERT · girdi doğrulama
2	3–4	Biçim dizisi · UAF · tamsayı/UB · hata/sinyal · statik analiz · sanitizer · fuzzing
3	5–7	Derleyici/OS korumaları · güvenli derleme hattı · kod gizlemeye giriş · proje

Kısa tarihçe — bellek hataları ve savunmaların yarışı

- 1988 — **Morris Worm** tampon taşmasını dünyaya duyurur
- 1996 — *Smashing the Stack for Fun and Profit* sömürüyü herkese öğretir
- 1998 → 2004 — **kanarya** (StackGuard) · **ASLR** (PaX) · **DEP/NX** · FORTIFY
- 2012–13 — **AddressSanitizer** ve **fuzzing** (AFL) hataları otomatik yakalar

Ders bu yarış izler: hatayı yaz → araç yakalasın → derleyici/OS korusun.

Bu hafta nereye oturuyor?

- **1–3. hafta:** güvenlik ilkeleri, tehdit modeli, kriptoloji.
- **Bu hafta (4):** C/C++ kodunu **sağlamlaştırmak** — hatasız yaz, hata kaçarsa zararı sınırla, okumayı zorlaştır.
- **5. hafta:** aynı konu Java/yorumlanan diller.

Öğrenme çıktısı

Bu hafta **ÖÇ.3** (ikili uygulama korumaları) üstünedir.

Sonunda yapabileceğiniz:

- Yaygın C/C++ zafiyetlerini tanımak ve düzeltmek
- Statik analiz, sanitizer, fuzzing kullanmak
- Derleyici/OS korumalarını açmak ve doğrulamak

Üç katman, doğru sıra

Kod sağlamlaştırma üç katmandır. Sıra önemli:

1. **Önce hatasız kod** (güvenli kodlama)
2. **Sonra zararı sınırla** (derleyici/OS korumaları)
3. **En son okumayı zorlaştır** (gizleme)



Gizleme, hatayı **düzeltilmez**.

Neden bu sıra?

- Hatalı koda koruma eklemek, çürük temele boya çekmek gibidir.
- Önce hatayı **kaldır**; sonra kaçanı **yakala**; en son tersine mühendisliği **yavaşlat**.
- Bu hafta üçünü de göreceğiz, bu sırayla.

0. Temel kavramlar (sıfırdan)

Bellek: yığın ve öbek

- **Yığın (stack):** fonksiyon çağrılarının yerel değişkenlerini tuttuğu, otomatik yönetilen bellek.
- **Öbek (heap):** `malloc` / `new` ile **elle** ayrılan, `free` / `delete` ile bırakılan bellek.

İkisi de taşma ve hata kaynağı olabilir.

İşaretçi (pointer)

- **İşaretçi:** bir bellek **adresini** tutan değişken.
- `p` bir adres; `*p` o adresteki değer.
- Yanlış adres → çökme ya da yanlış veri.

Tampon (buffer) ve taşma

- **Tampon:** ardışık bellek bloğu (ör. `char ad[16]`).
- **Taşma (buffer overflow):** tampona **sığmayan** kadar veri yazmak → komşu belleği bozar.
- Klasik ve tehlikeli bir hata sınıfı.

Neden taşma tehlikeli?

- Komşu değişkenleri, dönüş adresini bozabilir.
- Saldırgan bunu **denetim akışını** ele geçirmek için kullanabilir.
- Bu yüzden **sınır denetimi** hayatidir.

Tanımsız davranış (UB) nedir?

- **Tanımsız davranış (undefined behavior):** C/C++ standardının "sonucu belirsiz" dediği durumlar.
- Örnek: işaretli tamsayı taşması, dizinin dışına erişim.
- Derleyici bunu **istediği gibi** ele alabilir — hatta ilgili denetimi **silebilir**.

Derleyici bayrağı (flag)

- **Bayrak:** derleyiciye verilen seçenek.
- Örnek: `-O2` (optimizasyon), `-Wall` (uyarılar), `-fsanitize=address`.
- Doğru bayraklar birçok hatayı **derleme anında** yakalar.

Uyarı (warning) vs hata (error)

- **Hata:** derleme durur.
- **Uyarı:** derleme sürer ama bir sorun bildirilir.
- Kural: uyarıları **hataya çevir** (`-Werror`) — görmezden gelinen uyarı, gelecekteki açıktır.

CWE nedir?

- **CWE (Common Weakness Enumeration):** yazılım zayıflıklarının numaralı kataloğu.
- Örnek: CWE-416 = "use-after-free".
- Bir bulguyu CWE numarasıyla adlandırmak, onu **aranabilir** yapar.

CERT nedir?

- SEI CERT C/C++: güvenli kodlamanın **kural kitabı**.
- Her kural: hatalı örnek + uyumlu çözüm + risk + CWE bağı.
- Örnek: `STR31-C` = dizgeye yeterli yer ayır.

Statik vs dinamik analiz

- **Statik analiz:** kodu **çalıştırmadan** inceleme (derleyici uyarıları, clang-tidy).
- **Dinamik analiz:** kodu **çalıştırırken** izleme (sanitizer'lar).
- İkisi birbirini tamamlar.

Sanitizer nedir?

- **Sanitizer**: programı çalıştırırken bellek/UB hatalarını yakalayan derleyici aracı.
- Örnek: **ASan** (adres), **UBSan** (tanımsız davranış).
- Sürüme gitmez; **test/CI**'da kullanılır.

Fuzzing nedir?

- **Fuzzing:** programa **rastgele/beklenmeyen** girdiler verip çökme aramak.
- İnsanın düşünmediği girdileri bulur.
- Sanitizer'la birlikte çok güçlü.

ASLR, NX/DEP, kanarya

- **ASLR:** bellek adreslerini **rastgeleleştirir** (saldırgan adresi tahmin edemesin).
- **NX/DEP:** veri bölgesindeki baytları **kod olarak çalıştırmayı** engeller.
- **Yığın kanaryası:** dönüş adresinden önce bir **nöbetçi değer**; taşma onu bozarsa program durur.

Beyaz kutu saldırgan (hatırlatma)

- Programa sahip saldırgan (1. hafta MATE).
- Dizgeleri okur, fonksiyonları adıyla bulur, denetimi atlar.
- Gizleme bölümünde (bugün sonunda) buna döneceğiz.

İki saldırgan modeli — şema

Uzak saldırgan mı, cihazın sahibi mi?

aynı programa iki farklı tehdit modeli

UZAK SALDIRGAN

girdi gönderir
belleği göremez
bellek güvenliği + doğrulama yeter
1-5. haftalar

CİHAZIN SAHİBİ (MATE)

ikiliyi elinde tutar
hata ayıklayıcı bağlar
gizleme + RASP gerekir
6., 9., 14. haftalar

Gizlemeye buradan geçiyoruz: bellek güvenliği ikinci saldırgana karşı yetmez.

CI (sürekli entegrasyon)

- **CI (Continuous Integration):** her kod değişikliğinde otomatik derleme + test çalıştıran sistem.
- Güvenlik araçlarını (uyarı, statik analiz, sanitizer, fuzzing) **her birleştirmede** koşturur.

Şimdi hazırız

Terimler:

yığın/öbek · işaretçi · tampon/taşma · UB · bayrak · uyarı/hata · CWE · CERT · statik/dinamik ·
sanitizer · fuzzing · ASLR/NX/kanarya · CI

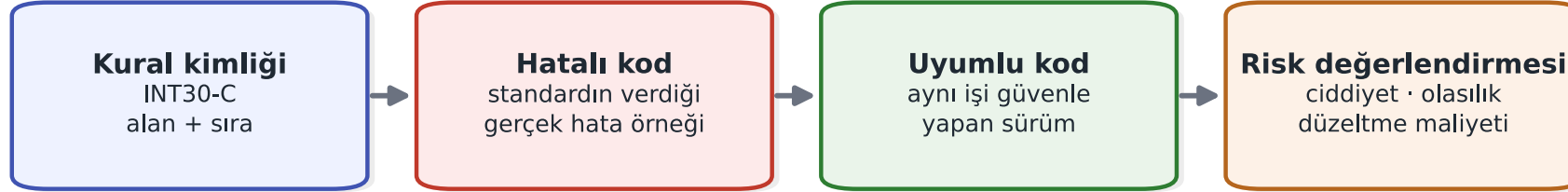
Şimdi: kod sağlamlaştırmanın katmanları.

1. Katmanlar ve SEI CERT

CERT kuralının anatomisi — şema

SEI CERT kuralının anatomisi

her kural aynı dört parçayla gelir



Kuralı ezberlemeyin: hatalı ile uyumlu kodun FARKINI okuyun; öğretici olan orasıdır.

Üç katman — şema

Kod sağlamlaştırmanın üç katmanı

yalnız en alttaki katman hatayı yok eder

3 · Kod gizleme

anlamayı zorlaştırır — hatayı SİLMEZ

2 · Derleyici / OS korumaları

sömürüyü zorlaştırır — hatayı SİLMEZ

1 · Güvenli kodlama

hatayı GERÇEKTEN ortadan kaldırır

Sıra: önce doğru yaz, sonra derleyiciyle sertleştir, en son gizle.

Katman 2 · Güvenli kodlama

- **Soru:** kodumda hata var mı?
- **Araçlar:** CERT kuralları, sanitizer, fuzzing.
- **Amaç:** hatayı en baştan **yazmamak**.

Katman 3 · Derleyici/OS

- **Soru:** hata kaçarsa sömürü zor mu?
- **Araçlar:** kanarya, FORTIFY, ASLR, NX, RELRO, CFI.
- **Amaç:** kalan hatanın **istismarını** pahalı kılmak.

Katman 4 · Gizleme

- **Soru:** kodu okuyan anlıyor mu?
- **Araçlar:** sembol/dize gizleme, günlük kaldırma, düzleştirme.
- **Amaç:** tersine mühendisliği **yavaşlatmak**.

Sahadan: şema gereksinimleri

Sertifikalı bir üründe:

- "Yaygın zafiyetler ele alınmalı: hassas veri, enjeksiyon, taşma, dile özgü zayıflıklar"
- "Savunmacı programlama **kodun tamamında tutarlı**"
- "Statik ve dinamik tersine mühendisliğe karşı koruma"

→ Yalnız native tarafta **18 sağlamlaştırma önlemi**.

SEI CERT: bir kural neye benzer?

Her CERT kuralının parçaları:

- **Kimlik:** STR31-C , MEM50-CPP
- **Başlık:** tek cümle
- **Hatalı örnek** ve **uyumlu çözüm**
- **Risk düzeyi** (L1/L2/L3) ve **CWE** bağı

Kural mı, öneri mi?

- **Kural (rule):** ihlali bir **zafiyettir**, denetlenebilir.
- **Öneri (recommendation):** iyi uygulama.
- İncelemeye **L1** (en yüksek risk) kurallarından başla.

Risk düzeyi nasıl belirlenir?

Risk = önem × olasılık × düzeltme maliyeti.

- **L1:** yüksek risk, önce bunlar.
- **L2 / L3:** giderek düşer.

CERT kategorileri (1)

Kısaltma	Kategori	Örnek
INT	Tamsayılar	INT32-C: işaretli taşmaya izin verme
ARR/STR	Dizi/dizge	STR31-C: yeterli yer ayır
MEM	Bellek	MEM30-C: bırakılmış belleğe erişme
FIO	Dosya G/Ç	FIO30-C: girdiyi biçim dizgesinden dışla

CERT kategorileri (2)

Kısaltma	Kategori	Örnek
ENV	Ortam	ENV33-C: <code>system()</code> çağırma
SIG	Sinyaller	SIG30-C: işleyicide yalnız güvenli fn.
ERR	Hata işleme	ERR33-C: kütüphane hatasını algıla
CON/MSC	Eşzamanlılık/çeşitli	CON43-C, MSC32-C

C++: MEM50-CPP, CTR50-CPP, STR50-CPP, EXP53-CPP, ERR50-CPP.

Otomatik CERT denetimi

- Uyarılar: `-Wall -Wextra -Wformat=2 -Wconversion`
- `clang-tidy cert-*`
- `cppcheck --addon=cert`
- MSVC `/analyze`

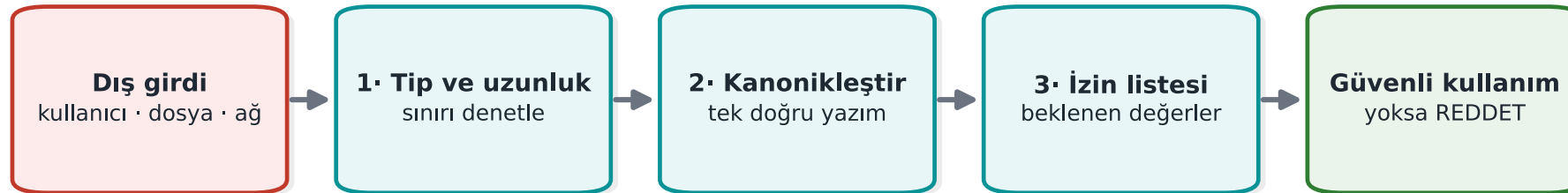
Değerlendirici önce bu taramayı çalıştırır, sonra elle okur.

Girdi doğrulama

Girdi doğrulama — şema

Güven sınırında girdi doğrulama

her dış girdi bu kapıdan geçer



Kara liste değil İZİN listesi: neyin yasak olduğunu değil, neyin serbest olduğunu yazın.

Girdi nereden gelir?

Sandığınızdan çok yerden. Ve **hepsi güvenilmez**:

- Komut satırı, ortam değişkenleri
- Dosya, ağ
- IPC (aynı makinedeki başka süreç)
- Veritabanı, kütüphane arayüzü (JNI)

Sık unutulan girdiler

- `argv[0]` bile saldırganın olabilir; `PATH` , `LANG` .
- Dosya biçimindeki **uzunluk alanları**.
- Ağ: "16 bayt" der, 1 bayt gönderir.
- "Kendi verimiz" sonradan değişmiş olabilir.

Girdi doğrulamanın beş ilkesi

1. **Beyaz liste** (izin verilenleri say), kara liste değil.
2. Önce **kanonikleştir**, sonra doğrula.
3. **Uzunluk** → **tür** → **aralık** → **biçim** sırasıyla.
4. **Güven sınırında**, en erken noktada.
5. **Reddet**, "temizleyip" kullanma.

İlke 1 · Beyaz liste

- **Kara liste:** "şunlar yasak" — hep bir şey unutulur.
- **Beyaz liste:** "yalnız şunlar serbest" — güvenli varsayılan.

Örnek: kullanıcı adı yalnız `[a-z0-9_]` olsun.

İlke 2 · Kanonikleştir

- Aynı şeyin farklı yazımları: `/tmp/../../etc` , `%2e%2e/` .
- Önce **tek biçime** indir (kanonikleştir), sonra denetle.
- Yoksa denetim atlatılır.

ilke 3 · `atoi` değil `strtol`

```
errno = 0;
long v = strtol(s, &son, 10);
if (son == s)      return -1; /* rakam yok */
if (*son != '\0')  return -1; /* "12abc" */
if (errno == ERANGE) return -1; /* taştı */
if (v < en_az || v > en_cok) return -1; /* iş kuralı */
```

Neden `atoi` tehlikeli?

- `atoi("abc")` = 0 (hata sessiz)
- `atoi("999999999999")` = **tanımsız** (taşma)
- `atoi("12abc")` = 12 (fazlalık yok sayılır)

`strtol` her hatayı **size bildirir**; `atoi` bildirmez.

İlke 3 · uzunluk önekli kayıt

```
if (kalan < 3) return -1;           /* başlık yok */
uint16_t uzunluk = tampon[1] << 8 | tampon[2];
if (uzunluk > kalan - 3) return -1; /* KAYNAK: bildirilen > gelen */
if (uzunluk > sizeof k->veri) return -1; /* HEDEF: sığmıyor */
memcpy(k->veri, tampon + 3, uzunluk);
```

İki ayrı denetim, doğru sıra

- **Kaynak denetimi:** bildirilen uzunluk, gelen veriden büyük mü?
- **Hedef denetimi:** hedef tampona sığıyor mu?
- Sıra: `kalan - 3` **sarmasın** diye önce `kalan < 3`.
- Sahada: JNI'den gelen her dizi native tarafta **yeniden** doğrulanır.

CERT çifti · STR31-C

```
strcpy(kopya, ad);                                /* HATALI */  
int n = snprintf(kopya, sizeof kopya, "%s", ad); /* UYUMLU */  
if (n < 0 || (size_t)n >= sizeof kopya) { /* kesildi */ }
```

`strcpy` sınır bilmez; `snprintf` boyutu bilir ve **kesilmeyi** bildirir.

CERT çifti · INT30-C

```
size_t kalan = toplam - okunan; /* HATALI: okunan>toplam → dev sayı */  
if (okunan > toplam) return HATA; /* UYUMLU: önce sırala */  
size_t kalan = toplam - okunan;
```

İşaretsiz çıkarma **sarar**; önce mantığı denetle.

CERT çifti · MEM30-C

```
for (d = bas; d; d = d->sonraki) free(d);          /* HATALI */  
while (d) { Dugum *s = d->sonraki; free(d); d = s; } /* UYUMLU */
```

`free(d)` 'den sonra `d->sonraki` okumak = bırakılmış belleğe erişim.

CERT çifti · ERR33-C

```
FILE *f = fopen(yol,"rb"); fread(t,1,n,f);          /* HATALI */  
if (!f) return HATA;  
if (fread(t,1,n,f) < n && ferror(f)) return HATA; /* UYUMLU */
```

Her dönüş değerini **denetle**; `fopen` NULL dönebilir.

Kural: bulguyu kimliğe bağla

Her bulguyu bir CERT kimliğiyle eşleştir:

"Bu satır MEM30-C ihlali (CWE-416)."

Böylece bulgu **nesnel** ve **aranabilir** olur.

Bölüm 1 — kısa sinama

1. Üç katman ve doğru sıra?
2. Kural ile öneri farkı?
3. `atoi` neden `strtol` 'a göre tehlikeli?

Bölüm 1 — cevaplar

1. **Sıra:** güvenli kod (kaynak) → **derleyici/araç** korumaları → **OS/çalışma zamanı** korumaları. Önce doğru yaz, sonra sertleştir, sonra çalışma anında savun.
2. **Kural** her zaman uygulanır, ihlali kusurdur; **öneri** bağlama göre iyi uygulamadır.
3. `atoi` hatayı **bildirmez** (taşma/geçersiz girdide tanımsız/0); `strtol` `endptr` + `ERANGE` ile **hatayı ve sınırı** kontrol ettirir.

2. Biçim dizisi açığı

Biçim dizisi açığı — şema

Biçim dizisi açığı

tek fark: biçim dizgesini kim belirliyor

printf(girdi)

girdi BİÇİM DİZGESİ sayılır

%x → yığını okur
%n → belleğe YAZAR
→ bilgi sızıntısı, kod akışı

printf("%s", girdi)

biçim dizgesi SABİT

girdi yalnız VERİ

güvenli

En tehlikelisi %n: yazdığı bayt sayısını belleğe yazar → seçili adrese yazma.

Sorun · `printf(girdi)`

```
printf(kullanici_girdisi);      /* HATALI */  
printf("%s", kullanici_girdisi); /* DOĞRU */
```

İlkinde kullanıcı girdisi bir **biçim dizgesi** olarak yorumlanır.

Neden tehlikeli?

`printf` biçim dizgesindeki `%` belirteçlerini **komut** sayar.

Kullanıcı `%x`, `%s`, `%n` gönderirse programa iş yaptırır.

Belirteç · %x , %p

- Yığındaki değerleri **okur** ve yazdırır.
- → **bilgi sızıntısı** (adresler, gizli değerler).
- ASLR'yi zayıflatır (adres sızarsa).

Belirteç · %s

- Yığındaki bir değeri **işaretçi** sanar ve o adresteki dizgeyi okur.
- Rastgele adres → çökme ya da bellek okuma.

Belirteç · %n

- Şimdiye kadar yazılan karakter sayısını bir adrese **yazar**.
- → belleğe **yazma** → bozulma, denetim ele geçirme.
- En tehlikelisi.

Aynı hata başka yerlerde

Sadece `printf` değil:

`syslog`, `fprintf`, `snprintf`, `err` / `warn` — hepsi biçim dizgesi alır.

Kural: biçim dizgesi **her zaman sabit** (FIO30-C).

Demo 1 — biçim dizisi

code/week-04/01-format-string · CWE-134

Adım	Ne görülür?
0	GCC <code>-Wformat-security</code> uyarır; MSVC <code>/analyze</code> yakalar
2 <code>%x.%x...</code>	Yığın dökülür, gizli (sentetik) değer görünür
3 <code>AAAA%n</code>	Linux korumasız çöker; Windows CRT <code>%n</code> 'i reddeder
4	<code>_FORTIFY_SOURCE : %n in writable segment</code>
5	Belirteçler yalnız metin olarak görünür

Düzeltilme · katman katman

1. Biçim dizgesi hep **sabit** (FIO30-C).
2. Uyarı: `-Wformat -Wformat-security -Werror=format-security`.
3. Kendi fonksiyonunuzu derleyiciye **denetlettirin**:

```
void gunluk_yaz(int duzey, const char *bicim, ...)  
    __attribute__((format(printf, 2, 3)));
```

4. Çalışma anı: `_FORTIFY_SOURCE=2` ; MS CRT `%n` kapalı.

3. Bellek: use-after-free

Yığın ve öbek — şema

Yığın (stack) ve öbek (heap)

taşmanın sonucu, belleğin hangi bölgesinde olduğuna göre değişir

YIĞIN

fonksiyonla açılır/kapanır
otomatik, hızlı
boyut derleme anında belli

taşma → dönüş adresi tehlikede

ÖBEK

malloc/free ile yönetilir
yaşam süresi size bağlı
boyut çalışma anında

taşma → komşu blok / meta veri

Kanarya yığını korur; öbek taşmasını yakalamaz — bu yüzden ASan gerekir.

Use-after-free — şema

Use-after-free zinciri

ASan raporundaki üç iz bu zincirin üç noktasıdır



Düzeltilme free ile use ARASINDADIR: $p = \text{NULL}$ ve tek sahiplik kuralı.

UAF nedir? · adım 1

- Bir nesne öbekte.
- **İki işaretçi** aynı nesneyi gösteriyor.

UAF · adım 2

- Biri `free` ediyor.
- Diğeri hâlâ o adresi tutuyor → **askıda (dangling)** işaretçi.

UAF · adım 3

- Ayırıcı aynı bloğu **başka nesneye** veriyor.
- Askıdaki işaretçi artık **yabancı veriyi** okur/yazar.
- Nesnede **fonksiyon işaretçisi** varsa (C++ sanal tablo) risk büyür.

UAF neden çok yaygın?

Tarayıcı/OS güncellemelerindeki "bellek bozulması düzeltildi" notlarının büyük kısmı bu sınıftır.

Karmaşık sahiplik → askıda işaretçi.

Demo 2 — UAF ve çift bırakma

code/week-04/02-kullanim-sonrasi · CWE-416/415 · MEM30-C

Adım	Ne görülür?
1	Serbest blok dolar → rol <code>user</code> → <code>admin</code>
2 ASan	heap-use-after-free + yerler
3	Çift <code>free</code> → tanımsız davranış
4 ASan	attempting double-free
5	<code>free</code> + <code>NULL</code> , tek sahip → güvenli

ASan raporu · üç yığın izi

```
ERROR: heap-use-after-free
#0 oturum_kullan uaf.c:48  <- 1. KULLANIM
freed here:
#1 oturum_kapat  uaf.c:31  <- 2. SERBEST BIRAKMA
allocated here:
#1 oturum_ac     uaf.c:22   <- 3. AYIRMA
```

Düzeltilme çoğunlukla **2. konumda**: sahiplik kararı orada yanlış.

Çözüm · C tarafı

- `free` 'den sonra işaretçiyi `NULL` yap.
- **Tek sahip**, tek bırakma noktası belirle.
- Sahiplik kuralını yaz: "bu belleği kim bırakır?"

Çözüm · C++ akıllı işaretçiler

İhtiyaç	C++
Tek sahip	<code>std::unique_ptr</code>
Paylaşılan	<code>std::shared_ptr</code>
Sahipsiz gözlem	<code>std::weak_ptr</code> → <code>lock()</code>
Dizi	<code>std::vector</code> , <code>std::array</code>

```
if (auto o = onbellek.lock()) kullan(*o); // yoksa eski belleğe erişilmez
```

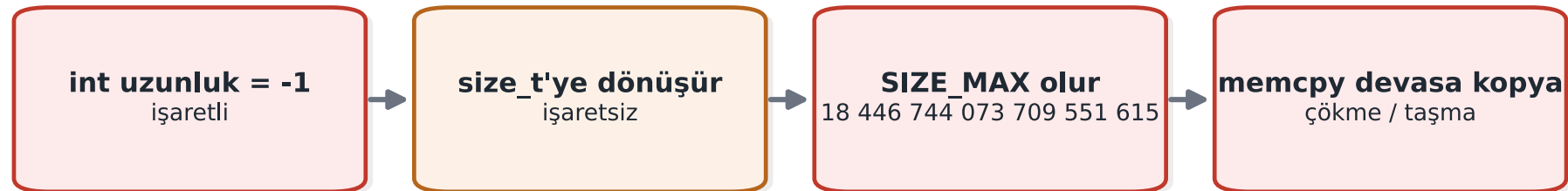
⚠ `get()` ham işaretçi; `vector` büyüyünce yineleyici geçersizleşir.

Tamsayılar ve UB

Tamsayı taşması — şema

-1 ne zaman dev olur?

işaretili → işaretsiz dönüşümü sessizce gerçekleşir



Kural: uzunlukları size_t ile tutun; dışarıdan gelen sayıyı hem alt hem üst sınırla denetleyin.

Dört tamsayı hatası

Hata	Örnek	Sonuç
İşaretsiz sarma	<code>0u - 1</code> = 4.294.967.295	Mantık hatası
İşaretli taşma	<code>INT_MAX + 1</code>	Tanımsız
Dönüşüm kaybı	<code>long long</code> → <code>int</code>	Kesilme
Kaydırma	<code>1 << 31</code>	Tanımsız

Neden önemli? · çarpım taşması

```
uint32_t boyut = adet * 4;    /* adet = 0x40000001 → boyut = 4 */  
dizi = malloc(boyut);        /* küçük blok */  
/* döngü adet kez yazar → öbek taşması */
```

Küçük ayrılan bloğa çok yazmak = taşma.

UB · derleyici denetimi silebilir

```
if (x + 100 < x) /* "taşarsa küçülür" sanısı */  
    return -1; /* işaretli taşma UB → derleyici bu dalı SİLEBİLİR */
```

- -00 'da "çalışır", -02 'de denetim **yok olur**.
- UB'ye güvenilmez.

Doğru: önce denetle

```
if ((b > 0 && a > INT_MAX - b) ||  
    (b < 0 && a < INT_MIN - b)) return false; /* taşma öncesi */  
if (__builtin_mul_overflow(a, b, &sonuc)) return false; /* GCC/Clang */  
/* C23: <stdckdint.h> ckd_add/ckd_mul · MSVC: <intsafe.h> */
```

Taşmayı **olmadan önce** yakala.

Demo 3 — UB ve UBSan

code/week-04/03-tanimsiz-davranis · CWE-190/758

- İşaretli taşma, geçersiz kaydırma, hizasız erişim
- x86'da çoğu **sessizce** yanlış sonuç
- `-fsanitize=undefined` → `signed integer overflow: 2147483647 + 1 ...`
- Windows: UBSan yok → `/RTC` , `/analyze` , CERT



`-fwrapv` mantık hatasını **gizler**; tanı için kullanma.

Hata işleme ve sinyaller

Sinyal işleyici — şema

Sinyal işleyicide ne yapılır?

sinyal, kesilen kodun ortasında gelebilir

YANLIŞ

printf, malloc, free
kilit alan her şey

yeniden girişte iç yapı bozulur
→ kilitlenme / çökme

DOĞRU

yalnız bayrak kur:
volatile sig_atomic_t

asıl işi ana döngü yapar
(write() gibi güvenli çağrılar)

Async-signal-safe olmayan fonksiyon çağırmak tanımsız davranıştır.

Denetlenmeyen dönüş değeri

Çağrı	Denetlenmezse
<code>malloc</code>	NULL'a yazma
<code>setresuid</code>	Yetkili kalır
<code>RAND_bytes</code>	"Rastgele" anahtar sıfır
<code>EVP_DecryptFinal_ex</code>	Kurcalanmış veri doğrulanmış sanılır

Hata işleme kuralları

1. Her dönüşü denetle (ERR33-C); `[[nodiscard]]`.
2. Hatada **güvenli duruma** dön (`goto temizle`) — **fail-closed**.
3. İleti ipucu vermesin: "kullanıcı yok" $\neq$ "parola yanlış".

Sahadan: opak dönüş değerleri

- Native fonksiyonlar ayrıntılı **hata kodu döndürmez**.
- Yalnız başarılı/başarısız, üstelik **opak** (9. hafta).
- Ayrıntılı kod = saldırgana "hangi denetimde takıldın" haritası.
- Tanı için ayrı, güvenli kanal → ödünleşim kaydı.

Sinyal işleyici · kural

```
static volatile sig_atomic_t durdur = 0;  
static void isleyici(int s){ (void)s; durdur = 1; } /* YALNIZ bayrak */
```

- İşleyicide `printf`, `malloc`, `free` **yok** (SIG30-C).
- Eşzamansız-güvenli olmayan fonksiyon çağırma.
- `signal()` yerine `sigaction()`.

Gerçek olay: regreSSHion

- 2024 CVE-2024-6387: zaman aşımı **sinyal işleyicisi** güvensiz bir günlük fonksiyonu çağırıyordu.
- Sonuç: uzaktan kod çalıştırma riski.
- Ders: işleyicide yalnız bayrak set et.

Bölüm 2–3 — kısa sinama

1. `%n` neden en tehlikelisi?
2. UAF raporundaki üç iz? Düzeltme hangisinde?
3. `if (x+100 < x)` neden silinebilir?
4. Sinyal işleyicide neden `printf` yok?

Bölüm 2–3 — cevaplar

1. `%n` biçim belirtecinin yazdığı **bayt sayısını belleğe YAZAR** → saldırgan seçtiği adrese yazıp akışı ele geçirebilir.
2. **allocated-by · freed-by · use-here**. Düzeltme **freed↔use** arasında: `free` sonrası işaretçiyi `NULL` yap / yaşam süresini düzelt.
3. İmzalı taşma **tanımsız davranıştır**; derleyici "taşma olmaz" varsayar → koşul hep `false` → **siler**. Kontrolü `__builtin_add_overflow` ile yapın.
4. `printf` **async-signal-safe değil**; işleyicide yeniden girişte kilit/durum bozular → kilitlenme/çökme. Yalnız `write` gibi güvenli çağrılar.

4. Statik analiz, sanitizer, fuzzing

ASan redzone — şema

AddressSanitizer neyi yakalar, neyi kaçıır?

nesneler arasına 'zehirli bölge' (redzone) koyar



ASan ayrıca hiç çalışmayan kod yolunu da göremez — fuzzing bu yüzden gerekir.

Statik ↔ dinamik — şema

Statik analiz ↔ dinamik analiz

biri diğerinin yerine geçmez — ikisi birden kullanılır

STATİK (çalıştırmadan)

tüm kod yollarına bakar
hiç çalışmayan yolu da görür

yanlış pozitif ÇOK
bağlamı bilmez

DİNAMİK (çalıştırarak)

yalnız yürütülen yol
gerçek hatayı KESİN görür

yanlış pozitif AZ
kapsam girdiye bağlı

Fuzzing üçüncü ayaktır: dinamik analizin göreceği YENİ yolları üretir.

Statik analiz nedir? (hatırlatma)

Kodu **çalıştırmadan** inceleyip hata arama.

Avantaj: çalışmayan yolları da görür.

Dezavantaj: **yanlış alarm** üretir.

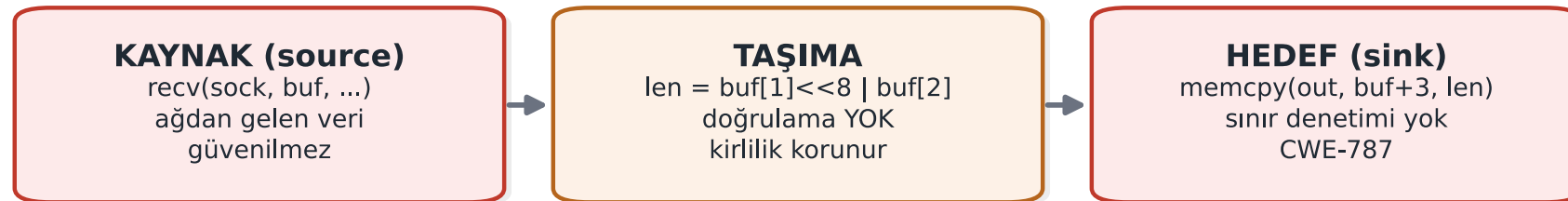
Statik analiz · katman katman

Düzyey	Araç	Bulduđu
Uyarılar	<code>-Wall -Wextra -Wconversion</code>	Dönüşüm, biçim
Analizci	<code>-fanalyzer , clang --analyze , /analyze</code>	NULL, sızıntı
Kural	<code>clang-tidy cert-* , cppcheck</code>	CERT ihlali
Anlamsal	CodeQL, Semgrep	Veri akışı
Ticari	Coverity, Klocwork	Derin analiz

Kirli veri: kaynak → hedef

Kirli veri takibi: kaynaktan hedefe

saldırganın yazdığı bir değer, denetlenmeden tehlikeli bir çağrıya ulaşıyor



Bulgu kuralı: kaynak ile hedef arasında doğrulama yoksa, o yol bir zafiyettir.

Araç, **doğrulanmamış** verinin tehlikeli bir yere aktığını görür.

Yanlış alarmla yaşamak

1. Önce **yüksek güvenilirlikli** kurallar.
2. Her bastırma **gerekçeli**: `// NOLINT(cert-err33-c): neden`.
3. Yeni kodda **sıfır uyarı** (CI zorlar).

Değerlendirici geliştiricinin kendi tarama raporunu ister.

Sanitizer nedir? (hatırlatma)

Programı **çalıştırırken** bellek/UB hatalarını yakalayan araç.

Yanlış alarm neredeyse **yok**; ama yalnız **çalışan yoldaki** hatayı bulur.

→ test + fuzzing ile birlikte.

Sanitizer ailesi

Sanitizer	Bulduğu	Bayrak
ASan	Taşma, UAF, çift bırakma	<code>-fsanitize=address</code>
LSan	Sızıntı	ASan ile
UBSan	Taşma, kaydırma, NULL	<code>-fsanitize=undefined</code>
MSan	İklendirilmemiş okuma	<code>-fsanitize=memory</code>
TSan	Veri yarışı	<code>-fsanitize=thread</code>

ASan nasıl çalışır? · gölge bellek

- Her 8 bayt için 1 **gölge bayt**: ne kadarı geçerli?
- Dizilerin önüne/arkasına **zehirli bölge** (redzone).
- Serbest bloklar **karantinada** bekler.
- Her erişimden önce gölge denetimi.

ASan'ın kör noktası

```
struct { char ad[8]; long yetki; } k;  
strcpy(k.ad, uzun_girdi); /* ad → yetki taşar */
```

Aynı yapının **içinde** alandan alana taşma → arada zehirli bölge yok → ASan **görmez** (1. hafta Demo 3).

Sanitizer sürüme gitmez

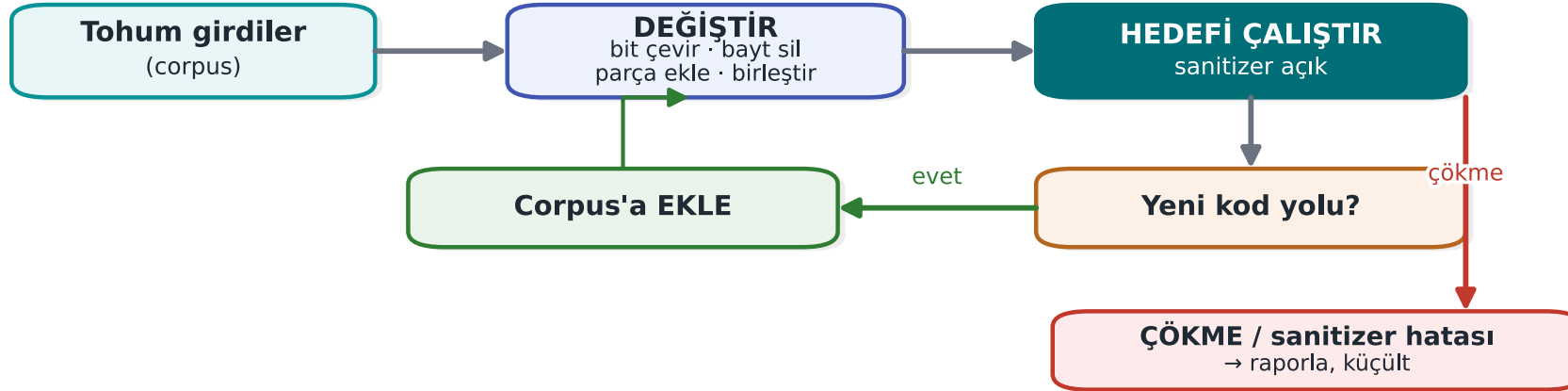
```
gcc -g -O1 -fno-omit-frame-pointer \  
-fsanitize=address,undefined p.c -o p_test
```

- Yavaşlatır ($\sim 2\times$), bellek kullanır.
- Yalnız **test/CI**'da; sürümde kapalı.

Fuzzing nedir? · kapsam güdümlü

Kapsam güdümlü fuzzing döngüsü

rastgele değil: yeni kod yolu bulan girdiyi saklar



İyi fuzz hedefi: hızlı · deterministik · kendine yeterli · dar girdi yüzeyi.

Yeni kod yolu açan girdi **saklanır** ve üzerinde çalışılır.

Fuzzing ne bulur?

- Sihirli baytları, doğru uzunluk alanını **kendiliğinden** bulur.
- İnsanın düşünmediği girdileri.
- Araçlar: libFuzzer, AFL++, honggfuzz, OSS-Fuzz (on binlerce hata).

Fuzz hedefi yazmak

```
int LLVMFuzzerTestOneInput(const uint8_t *veri, size_t boyut) {  
    ayristir(veri, boyut);    /* sınanacak fonksiyon */  
    return 0;  
}
```

```
clang -g -O1 -fsanitize=fuzzer,address fuzz.c ayristir.c -o f  
./f corpus/ -max_total_time=10
```

İyi fuzz hedefi

Dört özellik:

- **Hızlı** (ağ/disk yok)
- **Deterministik** (aynı girdi → aynı sonuç)
- **Durumsuz** (önceki çağrıdan etkilenmez)
- **Tek iş** (bir fonksiyonu sınar)

Demo 4 — fuzzing ile hata bulmak

code/week-04/04-fuzzing · CWE-125/787

Adım	Ne olur?
1	<code>tohum/normal.bin</code> sorunsuz
2	<code>libFuzzer</code> ≤ 10 sn → çökerten girdi
3	ASan ile yeniden oynat → tam rapor
4	Düzeltilmiş sürüm → çökme yok
5	<code>cokerten.bin</code> güvenle reddedilir

"Şu fonksiyon şu süre fuzzlandı" = güçlü kanıt (S16).

Bölüm 4 — kısa sinama

1. Statik ve dinamik analizin farkı?
2. ASan hangi taşmayı göremez?
3. İyi fuzz hedefinin dört özelliği?

Bölüm 4 — cevaplar

1. **Statik:** kodu çalıştırmadan tüm yolları inceler (yanlış pozitif olabilir). **Dinamik:** çalıştırırken yalnız yürütülen yolda **gerçek** hatayı gözler.
2. ASan **redzone** koyduğu yerleri görür; bir nesnenin **içindeki** alanlar arası taşmayı ve **hiç çalışmayan** yolları göremez.
3. **Hızlı, deterministik, kendine yeterli** (dış duruma bağlı değil), **dar bir girdi yüzeyi** işleyen; çökmede net başarısızlık veren hedef.

5. Derleyici ve OS korumaları

Derleyici/OS korumaları — şema

Derleyici ve işletim sistemi korumaları

saldırgan yukarıdan aşağı her katmanı ayrı ayrı aşmak zorunda

Yığın kanaryası	dönüş adresi ezilirse programı durdurur
ASLR	adresler her çalıştırmada değişir
DEP / NX	veri sayfası çalıştırılmaz
RELRO · PIE · CFI	GOT yazılamaz, çağrı akışı denetlenir
MANTIK HATASI	hiçbir koruma yakalamaz

Bu katmanlar sömürüyü zorlaştırır, HATAYI SİLMEZ — hatayı güvenli kodlama siler.

Katman 3 · fikir

Hata kaçtı diyelim.

- Sömürüyü **pahalı** kılan otomatik korumalar var.
- Çoğu bir **derleme bayrağıyla** açılır.
- Bedava değil ama ucuz.

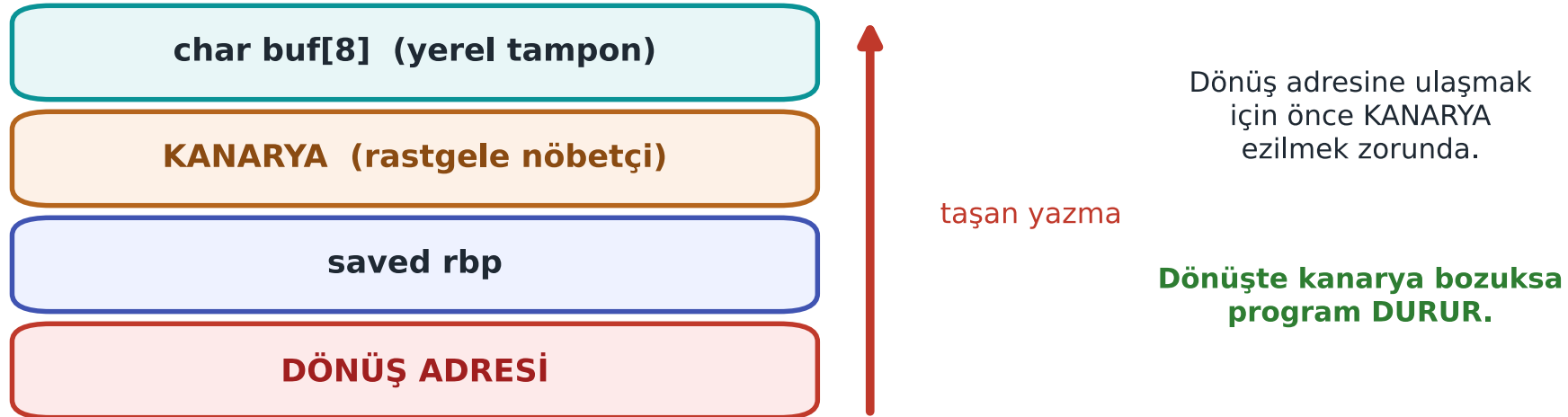
Yığın kanaryası

- Dönüş adresinden önce bir **nöbetçi değer** konur.
- Taşma bu değeri bozarsa, dönüşten önce fark edilir ve program **durur**.
- `-fstack-protector-strong` / MSVC `/GS` .

Yığın taşması ve kanarya — şema

Yığın taşması ve kanarya

taşma tampondan dönüş adresine doğru yazar



Durdurmaz: hedefli yazma · öbek (heap) taşması · bilgi sızıntısı.

FORTIFY_SOURCE

- Boyutu bilinen tampona fazla yazan **kütüphane çağrılarını** yakalar.
- `-D_FORTIFY_SOURCE=2` (+ en az `-O1`).
- Kendi döngülerinizi **korumaz**.

PIE + ASLR

- **ASLR:** adresleri rastgeleleştirir.
- **PIE:** programın da rastgele yüklenmesini sağlar.
- `-fPIE -pie / /DYNAMICBASE /HIGHENTROPYVA`.

DEP/NX ve RELRO

- **DEP/NX:** veri bölgesini çalıştırılmaz yapar (varsayılan).
- **RELRO:** GOT gibi tabloları salt-okunur yapar → `-Wl,-z,relro,-z,now` .

CFI / CFG ve gölge yığın

- **CFI/CFG:** dolaylı çağrıların yalnız **geçerli** hedeflere gitmesini denetler.
- `-fcf-protection` , `-fsanitize=cfi` / `/guard:cf` .
- **Gölge yığın:** dönüş adreslerinin ikinci kopyası (CET).

Korumalar · tek tablo

Koruma	GCC/Clang	MSVC
Kanarya	<code>-fstack-protector-strong</code>	<code>/GS</code>
FORTIFY	<code>-D_FORTIFY_SOURCE=2</code>	Güvenli CRT
PIE/ASLR	<code>-fPIE -pie</code>	<code>/DYNAMICBASE</code>
NX	varsayılan	<code>/NXCOMPAT</code>
RELRO	<code>-Wl,-z,relro,-z,now</code>	—
CFI	<code>-fcf-protection</code>	<code>/guard:cf</code>



Her koruma sınırlıdır

Koruma	Durdurur	Durdurmaz
Kanarya	Dönüş adresine taşma	Öbek taşması, sızan kanarya
FORTIFY	Kütüphane çağrısı taşması	Kendi döngüleriniz
ASLR	Sabit adres varsayımı	Adres sızıntısı
NX	Veri bölgesinde kod	Kod yeniden kullanımı (ROP)

Ana kural

Her koruma bir **teknîği** zorlaştırır, bir **hata sınıfını** kaldırmaz.

Mantık hatasına **hiçbiri** yetmez → önce güvenli kodlama.

Demo 5 — korumaları açıp kapatmak

code/week-04/05-derleyici-korumalari · CWE-121

- Aynı taşma: `giris_zayif` (kapalı) vs `giris_sert` (açık).
- Sert: `*** stack smashing detected ***` / Windows `0xC0000409`.
- Zayıf: sessiz bozulma ya da çökme.

```
readelf -h p | grep Type          # DYN = PIE
readelf -d p | grep BIND_NOW      # tam RELRO
readelf -s p | grep __stack_chk   # kanarya
```

Önerilen sürüm bayrakları

GCC/Clang:

```
-O2 -D_FORTIFY_SOURCE=2 -fstack-protector-strong -fPIE -pie -Wl,-z,relro,-z,now -fcf-protection -Wall -Wextra -Wformat=2 -Werror=format-security
```

MSVC:

```
/O2 /GS /sdl /guard:cf /DYNAMICBASE /HIGHENTROPYVA /NXCOMPAT /CETCOMPAT /W4
```

Koruma tablosu = kanıt

- Değerlendirici her ikili ve `.so` için **koruma tablosu** çıkarır.
- Kapalı koruma bir **bulgudur**; gerekçesi ödünleşim kaydında olmalı.
- Şema: uygulama, korumaların açık olduğunu **kendisi** de denetleyebilir.

Güvenli derleme hattı (CI)

Güvenli derleme hattı (CI)

her adım geçmeden bir sonrakine geçilmez



Ucuzdan pahalıya: erken adımlar kolay hataları eler, pahalı adım yalnız kalanlara harcanır.

Her hatalı adım birleştirmeyi **durdurur**.

CI · başarısızlık ölçütü

Aşama	Başarısızlık
Derleme	Tek uyarı
Sanitizer'lı test	Herhangi bir rapor
Fuzzing	Yeni çökme (eski corpus = regresyon)
İkili denetimi	Günlük dizgesi, sembol, kapalı koruma

Dersin CMake kipleri

Kip	Bayraklar
korumasiz	-00
optimize	-02
asan	-01 -fsanitize=address
denetimli	-02 -D_FORTIFY_SOURCE=2
guvenli	-02 + düzeltilmiş kaynak

Koruma bayraklarının çoğu: Demo 5 `giris_sert`.

Derleme ortamı da hedeftir

- Erişim kaydı; bağımlılık **sabitleme** (5. hafta SBOM).
- İmzalı sürüm; **yeniden üretilebilir** derleme.
- Derleme makinesi ele geçerse tüm ürün etkilenir.

Bölüm 5 — kısa sinama

1. Kanarya neyi durdurur, neyi durdurmaz?
2. ASLR neden bir bilgi sızıntısıyla zayıflar?
3. Neden "hiçbir koruma mantık hatasına yetmez"?

Bölüm 5 — cevaplar

1. **Durdurur:** dönüş adresini ezen **ardışık** yığın taşmasını (kanarya bozulu). **Durdurmaz:** hedefli yazma, heap taşması, bilgi sızıntısı.
2. Tek bir **sızan adres** modülün taban adresini açığa çıkarır (offset sabit) → tüm rastgelelik çöker.
3. Korumalar **bellek** ihlalini yakalar; yanlış yetkilendirme/iş kuralı **geçerli** bellek erişimidir → hiçbir koruma tetiklenmez. Doğru tasarım + inceleme gerekir.

6. Kod gizlemeye giriş

Katman 4 · neden?

- Beyaz kutu saldırgan: dizgeleri okur, fonksiyonları adıyla bulur, denetimi atlar.
- Gizleme: davranış aynı, **anlaşılması zor**.

Gizlemenin amacı

İmkânsız kılmak değil, **pahalı** kılmak (Cookbook 12.1):

- maliyet > korunan değer
- otomasyonu zorlaştır (çeşitlendirme, 14. hafta)
- diğer katmanlara **zaman** kazandır

Tek gelişmiş teknik yerine **birkaç basit teknik + veri gizleme**.

Gizleme haritası

Aile	Neyi gizler?	Hafta
Düzen	Adlar, yapı	4
Veri	Sabit, dize, değişken	4, 9
Kontrol akışı	Algoritma yapısı	4, 9
Önleyici	Analiz aracı	9
Sanallaştırma	Makine kodu	9, 14

Sahadan: native tarafta 18 önlem

Sembol görünürlüğü kapalı · ad anlamsızlaştırma · sabit aritmetik gizleme · dize şifreleme (kullanınca çöz, **hemen sil**) · opak boolean · kendi kütüphane fonksiyonları · sahte işlem/ölü dal · düzleştirme + rastgele çıkış · sürümde günlük **yok**.

➔ Güç **birlikten** ve RASP'tan.

Üç ucuz önlem

Önlem	Nasıl?
Sembolleri gizle	<code>static</code> , <code>-fvisibility=hidden</code> , <code>strip -s</code>
Günlüğü kaldır	Makroyu derlemede boşalt
Dizgeleri gizle	Derlemede şifrele, kullanınca çöz, hemen sil

Günlük makrosu

```
#ifndef GUNLUK_ACIK
#  define GUNLUK(...) fprintf(stderr, __VA_ARGS__)
#else
#  define GUNLUK(...) ((void)0)  /* dizge de çağrı da yok */
#endif
```

Sürümde `GUNLUK_ACIK` tanımlı değil → günlük **hiç derlenmez**.



Çalışma anı bayrağı yetmez

```
if (hata_ayikla) printf("..."); /* KÖTÜ */
```

- Dizge ikili dosyada **kalır**.
- Saldırgan bayrağı değiştirip günlüğü **yeniden açar**.
- Doğrusu: derleme anında **çıkart**.

Dize gizleme $\neq$ anahtar saklama

- Çözülen dizge kullanım anında **bellekte açık**.
- Çözme anahtarı **programın içinde**.
- Durdurduğu: `strings` gibi **statik** taramalar.
- Çalışırken inceleyene $\rightarrow$ RASP (6). Gerçek anahtar $\rightarrow$ whitebox (11).

Demo 6 — sembol ve dize sızıntısı

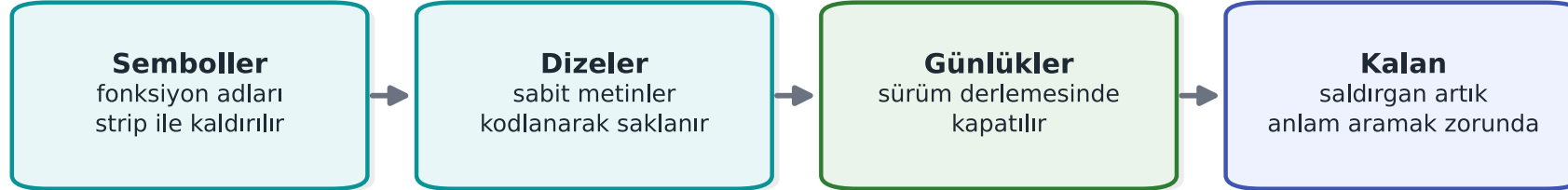
code/week-04/06-sembol-dize · CWE-200/215

Adım	Ne olur?
1	<code>gizli_acik</code> ve <code>gizli_kapali</code> aynı sonuç
2 <code>strings</code>	Açıkta lisans dizgesi ve <code>[LOG]</code> ; kapalıda yok
3 <code>nm</code>	Açıkta <code>lisans_dogrula</code> ; kapalıda sembol yok
4	Windows: adlar PDB'de; kapalı PDB üretmez

Sembol ve dize gizleme — şema

Sembol, dize ve günlük gizleme: ilk savunma hattı

tersine mühendisliğin en ucuz adımını pahalılaştırmak



En ucuz koruma budur ve çoğu projede eksiktir: strings çıktınızı bir kez okuyun.

Kontrol akışı düzeltme

```
int durum = 1;
for (;;) switch (durum) {
    case 1: durum = 2; break;
    case 2: durum = (kosul ? 3 : 4); break;
    case 3: durum = 5; break;
    case 4: return BASARISIZ;
    case 5: return BASARILI;
}
```

Blokların doğal komşuluğu kaybolur; sıra yalnız **durum değişkeninde**.

Düzleştirme: önce/sonra — şema

Kontrol akışı düzleştirme ve opak değerler

aynı davranış, okunamaz grafik

ÖNCE

if / else / while
grafik okunabilir
algoritmanın iskeleti görünür
birkaç temel blok

SONRA

tek switch + durum değişkeni
sıra değişkende saklı
opak yüklem sahte dal ekler
çok sayıda blok

Maliyeti vardır: boyut ve hız artar. 9. ve 14. haftada ölçmeyi öğreneceğiz.

Düzleştirmeyi güçlendirenler

- **Durum değerlerini gizle** (aritmetik dönüşüm)
- **Sahte/ölü bloklar** (hangisi gerçek?)
- **Rastgele çıkış** (hata nerede yakalandı, okunmaz)
- **Opak değerler** (tek bayt çeviremez)

Tek başına şablon kısa sürede çözülür → 9. hafta derinlik.

Demo 7 — düzleştirme

code/week-04/07-akis-duzlestirme

Adım	Ne olur?
1	<code>pin.c</code> ve <code>pin_duz.c</code> aynı sonuç (PIN sentetik)
2	<code>objdump</code> : düzleştirilmişte çok daha fazla dal
3	Süre: düzleştirmenin maliyeti

→ 14. hafta Tigress `Flatten` ile otomatik; 9. hafta etkinliği ölçmek.

Kavram: 9. haftaya bağlanan üç teknik

- **Fonksiyon adı gizleme:** dışa açık adlar anlamsız; okunabilirlik makroyla.
- **Bellek ayırma gizleme:** "32 bayt = anahtar" izini bulanıklaştır.
- **Dinamik şifreleme:** hassas kod ikilide şifreli; DEP/NX ile çatışır, seçici.

Bölüm 6 — kısa sinama

1. Gizleme hatayı düzeltir mi?
2. Günlüğü çalışma anı bayrağıyla susturmak neden yetmez?
3. Düzleştirmeyi güçlendiren üç teknik?

Bölüm 6 — cevaplar

1. **Hayır.** Gizleme yalnız tersine mühendisliği zorlaştırır; açık yerinde durur. **Önce düzelt, sonra gizle.**
2. Dize/kod ikili dosyada kalır; saldırgan bayrağı çevirir/dizeleri okur. Sürümde log'u **derleme zamanında** çıkarın (makro/ölü kod eleme).
3. **Opak yüklemeler + sahte bloklar/ölü dallar + durum değişkeni şifreleme & rastgele çıkış** (ayrıca ad/dize gizleme).

7. Proje ve kapanış

Proje · S9 "Kod sağlamlaştırma" — 1

- [] Önerilen sürüm bayraklarıyla derle; koruma tablosunu ekle (`checksec` / `dumpbin`).
- [] `clang-tidy` / `cppcheck` ile CERT taraması; ≥ 5 bulguyu düzelt (kural kimliğiyle).

Proje · S9 — 2

- [] Testleri ASan + UBSan ile çalıştır.
- [] En az bir fuzz hedefi, ≥ 10 dk.
- [] Sürümde günlük yok; hassas dizge `strings` çıktısında yok.

Önlem kartı: Açıklama → Uygulama → Doğrulama → Kalan risk.

Çözümlü kendini sinama

Soru 1

Üç katmandan hangisi hatayı gerçekten kaldırır?

Cevap: Yalnız **güvenli kodlama** (katman 2). Derleyici/OS korumaları istismarı zorlaştırır; gizleme okumayı yavaşlatır — ikisi de hatayı **kaldırmaz**.

Soru 2

`printf(girdi)` 'nin en hafif sonucu nedir?

Cevap: Bilgi sızıntısı (`%x` / `%p` ile yığın okuma). En ağırı `%n` ile belleğe yazma.

Soru 3

UAF raporundaki üç yığın izi? Düzeltme hangisinde?

Cevap: Kullanım, serbest bırakma, ayırma. Düzeltme çoğunlukla **serbest bırakma** noktasında (sahiplik kararı).

Soru 4

`if (x + 100 < x)` neden silinebilir?

Cevap: İşaretli tamsayı taşması **tanımsız davranıştır**; derleyici "taşma olmaz" varsayıp denetimi optimize edip **silebilir**.

Soru 5

Sinyal işleyicide neden `printf` yok?

Cevap: `printf` / `malloc` eşzamansız-güvenli değildir; işleyici bunları kesintiye uğramış bir durumda çağırırsa bozulma olur. Yalnız bir bayrak set edilir (SIG30-C).

Soru 6

ASan hangi taşmayı göremez?

Cevap: Aynı yapının **içinde** alandan alana taşmayı (aralarında zehirli bölge yoktur).

Soru 7

İyi bir fuzz hedefinin dört özelliği?

Cevap: Hızlı, deterministik, durumsuz, tek iş.

Soru 8

Kanarya neyi durdurur, neyi durdurmaz?

Cevap: Durdurur: dönüş adresine ulaşan yığın taşması. Durdurmaz: öbek taşması, önceki değişkenlerin bozulması, sızan kanarya.

Soru 9

ASLR neden bir bilgi sızıntısıyla zayıflar?

Cevap: Bir adres sızarsa, saldırgan diğer adresleri ona göre **hesaplayabilir**; rastgelelik avantajı kaybolur.

Soru 10

Günlüğü çalışma anı bayrağıyla susturmak neden yetmez?

Cevap: Dizge ikili dosyada **kalır** ve saldırgan bayrağı değiştirip günlüğü yeniden açabilir. Derleme anında çıkarılmalı.

Soru 11

Düzleştirmeyi güçlendiren üç teknik?

Cevap: Durum değerlerini gizleme, sahte/ölü bloklar, rastgele çıkış noktası (ve opak değerler).

Soru 12

Dize gizleme neden anahtar saklama değildir?

Cevap: Çözülen dizge kullanımda bellekte açıktır ve çözme anahtarı programın içindedir; yalnız statik taramayı durdurur.

Özet: bu haftanın tek cümlesi

Önce **hatasız kod** (CERT + sanitizer + fuzzing), sonra **zararı sınırla** (derleyici/OS korumaları), en son **okumayı zorlaştır** (gizleme) — ve hiçbir koruma güvenli kodlamanın yerini tutmaz.

Gelecek hafta

5. hafta — Kod sağlamlaştırma: Java ve yorumlanan diller

Enjeksiyon (SQL, komut, yol), ProGuard/R8 ile gizleme, bağımlılık güvenliği ve SBOM.

Hazırlık: JDK 17+, Maven · `code/week-05`.

Ek A · Baştan sona bir açık

Hatalı kod

```
void selamla(const char *ad) {  
    char tampon[16];  
    strcpy(tampon, ad);          /* sınır yok */  
    printf("Merhaba %s\n", tampon);  
}
```

ad 16 bayttan uzunsa ne olur?

Ne oluyor? · adım adım

- `tampon` yığında 16 bayt.
- `strcpy` `ad` 'ı **sonuna kadar** kopyalar, sınıra bakmaz.
- 16'dan uzun `ad` → komşu bellek (dönüş adresi dahil) bozulur.

Saldırı · sonuç

- Kısa taşma: komşu değişken bozulur (mantık hatası).
- Uzun taşma: **dönüş adresi** bozulur → çökme ya da denetim ele geçirme.
- Korumasızsa: ciddi açık (CWE-121).

Düzeltilme 1 · sınırlı kopyalama

```
int n = snprintf(tampon, sizeof tampon, "%s", ad);  
if (n < 0 || (size_t)n >= sizeof tampon) {  
    /* ad kesildi: reddet ya da işaretle */  
}
```

`snprintf` boyutu bilir ve **kesilmeyi bildirir**.

Düzeltilme 2 · katmanlar

- **Kodlama:** `snprintf` (STR31-C).
- **Derleyici:** `-fstack-protector-strong` (kanarya).
- **FORTIFY:** `-D_FORTIFY_SOURCE=2` kütüphane çağrısını denetler.
- **Test:** ASan uzun girdiyle taşmayı yakalar.

Ders: tek satır, çok katman

- Bir hata: `strcpy` .
- Savunma: doğru fonksiyon + derleyici koruması + sanitizer testi.
- Ama **asıl** çözüm ilk satırda: sınırlı kopyalama.

Ek B · Sık yapılan hatalar

Hata · uyarıları görmezden gelmek

- "Sadece uyarı" diye geçilen şey, yarınki açıktır.
- `-Werror` ile uyarıyı hataya çevir.

Hata · dönüş değerini denetlememek

- `malloc` , `fread` , `RAND_bytes` NULL/hata dönebilir.
- Denetlenmezse sessiz felaket (ERR33-C).

Hata · sanitizer'ı sürüme koymak

- Sanitizer test/CI içindir; yavaş ve bilgi sızdırır.
- Sürüm derlemesinde **kapalı**.

Hata · `-fwrapv` ile UB'yi gizlemek

- Taşmayı "tanımlı" yapmak mantık hatasını **gizler**.
- Tanı için UBSan kullan, düzelt.

Hata · gizlemeyle hatayı örtmek

- Gizleme hatayı **düzeltilmez**, sadece saklar.
- Önce düzelt, sonra gizle.

Hata · girdiyi "temizleyip" kullanmak

- Tehlikeli girdiyi düzeltmeye çalışmak yerine **reddet**.
- Beyaz liste + reddetme en güvenlisi.

Ek C · Hızlı başvuru

Sözlük

Terim	Anlam
UB	Tanımsız davranış
UAF	Kullanım-sonrası-serbest
CERT/CWE	Kural kitabı / zayıflık kataloğu
Sanitizer	Çalışma anı hata dedektörü
Fuzzing	Rastgele girdiyle çökme arama
Kanarya/ASLR/NX	Derleyici/OS korumaları

Sağlamlaştırma kontrol listesi

- ☐ -Werror ve CERT taraması temiz
- ☐ Her dönüş değeri denetli
- ☐ Testler ASan + UBSan ile geçiyor
- ☐ En az bir fuzz hedefi çalıştı
- ☐ Sürüm bayrakları açık (koruma tablosu)
- ☐ Sürümde günlük/hassas dize yok

Son söz (4. hafta)

Güvenli kod bir kez yazılmaz; **her derlemede** araçlarla denetlenir.

Önce doğru kod, sonra koruma, en son gizleme. Sıra önemlidir.